

Memory-Centric Agentic Inference — cycles 4-6

2026-05-11

Contents

Memory-Centric Agentic Inference — cycles 4-6	1
Abstract	1
Introduction	1
Methodology	2
Results	2
Discussion	8
Conclusions and Recommendations	10
References	10
Appendix: Implementation Details	10

Memory-Centric Agentic Inference — cycles 4-6

Abstract

Cycles 4-6 turned the earlier taxonomy, lifetime model, and cost model into an executable policy comparison, a scheduling-abstraction evaluation, and a concrete architecture proposal. The work narrowed the central thesis rather than broadening it: memory-centric architecture is not justified as a universal replacement for conventional LLM serving, but it is supported in synthetic agentic regimes where non-KV objects carry reuse value, correctness risk, branch survival, verifier delay, or durable workspace dependencies.

Cycle 4 built M-SIM-1, a deterministic synthetic workload generator and memory-policy comparison harness. It compared four policies across six workload regimes and showed that controls selected an HBM-first baseline, RAG selected reuse-aware tiering, and code-agent, verification-heavy, and branch/merge workloads selected branch/verifier/durable-aware placement. Cycle 5 built M-SCHED-1, which treated scheduling units as information boundaries and found that controls prefer coarse model/cache-page scheduling, RAG prefers memory-object scheduling, and deeper agentic regimes prefer trajectory/DAG scheduling. Cycle 6 built M-ARCH-1, a three-option architecture package: conventional serving, memory-object-aware runtime, and trajectory/DAG-aware memory fabric.

The cycle 6 audit validated the architecture package after one scoped fix: `architecture_options.csv` now lists explicit supported object sets for each architecture option instead of relying only on observed dominant scheduler outputs. The remaining agenda is not repair; it is calibration and prototyping with real traces, measured tier constants, queueing overhead, compression costs, and implementation benchmarks.

Introduction

The original research question asks whether agentic LLM infrastructure should be designed around memory movement, placement, reuse, compression, and lifetime management, rather than arithmetic throughput alone. Cycles 1-3 established the prerequisites: a workload and

memory-object taxonomy (M-TAX-1), a lifetime and reuse-distance model (M-LIFE-1), and a heterogeneous memory cost model (M-COST-1). Cycles 4-6 used those prerequisites to test policy behavior, scheduler visibility, and architecture boundaries.

A memory object means a first-class unit of state such as weights, KV cache, prefix cache, retrieved context, tool output, intermediate scratch, branch state, verifier state, trajectory log, durable workspace, or semantic cache entry. A scheduling unit means the boundary at which the system makes placement or retention decisions, such as a request, job, kernel, model, cache page, context segment, memory object, or agent trajectory graph. A trajectory/DAG is the graph of agent actions, branches, verifier loops, merges, discards, replay points, and durable artifacts.

The important result of these cycles is a boundary condition. Conventional serving remains appropriate when the useful state is mostly weights, active KV cache, prefix cache, and transient scratch. Memory-object visibility becomes useful when retrieved context, semantic cache entries, and tool outputs have reusable value or provenance requirements. Trajectory/DAG visibility becomes useful when branch state, verifier state, trajectory logs, and durable workspace state determine whether eviction causes recomputation, lost evidence, or correctness loss.

Methodology

The cycle sequence followed the milestone plan in `plan_of_record.md`.

Cycle 4 researcher session 89a82512-72e9-4051-bc4e-79e8086a63db specified M-SIM-1: build an executable workload generator and policy comparison harness using the validated taxonomy, lifetime model, and cost model. Worker session 8a332843-0be4-4bca-8d84-85f22b215bd5 produced the simulator artifacts. Auditor session 87c49730-6591-4fac-a9f5-62f2630baf6a validated the milestone after patching a moderate issue: the simulator was checking for `energy_proxy_score` and `dollar_proxy_score` fields without actually using them in scoring. After the patch, those cost-model proxy fields were threaded into simulator scoring.

Cycle 5 researcher session ceac8c77-51f8-4162-89ee-72b733822019 specified M-SCHED-1: evaluate scheduling abstractions using the simulator outputs. Worker session 5468f214-6855-4f34-8aad-1613ec95000e produced scheduling comparison artifacts. Auditor session eb34f7da-0f9f-498f-9b3a-37bf33ec243e validated the package with no moderate findings.

Cycle 6 researcher session 7031eb0d-9bee-407f-bc44-57409388096b specified M-ARCH-1: synthesize the validated milestones into architecture options, runtime/compiler hooks, policy matrices, failure modes, and a ranked research agenda. Worker session 94a84a42-0e3f-4bf8-8f4f-184625a04097 produced the architecture package. Auditor session c8733e5d-7874-4e7c-aae1-e679c3897573 validated it after patching one moderate issue in the architecture option generation.

All numerical values in these cycles are synthetic normalized scores unless explicitly labeled otherwise. The artifacts do not claim production traces, measured hardware constants, measured prices, queueing behavior, compression overheads, or vendor-specific capacities.

Results

Cycle 4: Workload Generator and Policy Comparison

Cycle 4 built `memory-centric-agentic/simulator_design.md`, `scripts/simulate_memory_policies.py`, `scripts/plot_sim_policy_results.py`, and six generated outputs: `sim_workload_events.csv`, `sim_policy_results.csv`, `sim_policy_object_breakdown.csv`, `sim_special_cases.csv`, `sim_policy_results.png`, and `sim_object_breakdown.png`.

The simulator generated 215 synthetic events across six regimes: single-turn chat control, batch/offline control, RAG with retrieved-context reuse, code-agent loop with tool outputs and durable workspace, verification-heavy agent, and multi-agent branch/merge run. Each event carried object class, size, reuse probability, branch fanout, branch survival, verifier delay, durability horizon, correctness sensitivity, recompute cost, and loss cost.

The four compared policies were:

- `hbm_first_baseline`: prioritizes weights, KV cache, and prefix cache.
- `reuse_aware_tiering`: prioritizes reuse probability and recomputation avoidance.
- `branch_verifier_durable_aware`: adds branch survival, verifier delay, durability horizon, and correctness-sensitive loss.
- `cost_proxy_balanced`: blends retained value, movement, energy proxy, dollar proxy, and risk.

The validated result was regime-dependent. The two controls selected `hbm_first_baseline` and were labeled weakened for the memory-centric thesis. RAG selected `reuse_aware_tiering` and was labeled ambiguous, because its value came from context and semantic-cache reuse rather than branch or durable trajectory state. The code-agent, verification-heavy, and branch/merge regimes selected `branch_verifier_durable_aware` and were labeled strengthened.

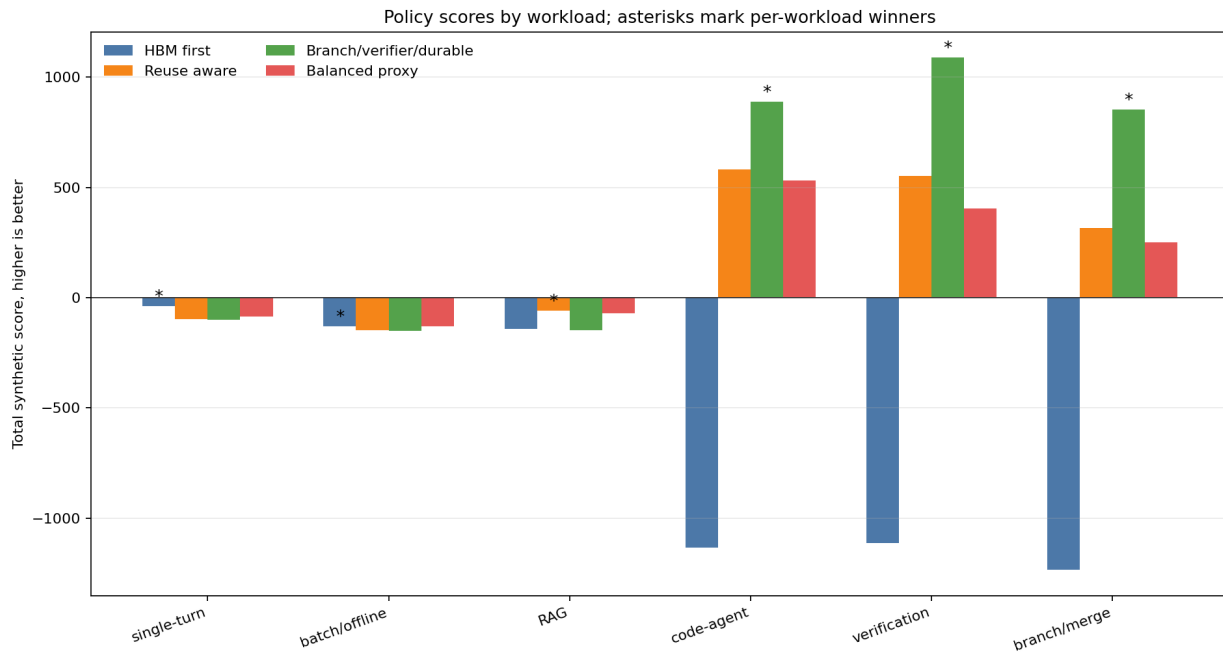


Figure 1: Simulator policy comparison across workload regimes. The controls favor HBM-first serving, RAG favors reuse-aware tiering, and deeper agentic regimes favor branch/verifier/durable-aware placement.

The object breakdown explained why the wins were not KV-only. In the code-agent regime, the strengthened policy was driven by durable workspace, tool output, branch state, and trajectory log contributions. In verification-heavy and branch/merge regimes, verifier state and trajectory-related objects dominated. This supported the narrowed claim that non-KV state can be causal in agentic workloads.

The simulator also emitted special cases. Equal tier costs collapsed policy differences to the HBM-first baseline with a zero score spread. Zero reuse, zero branch survival, zero durable

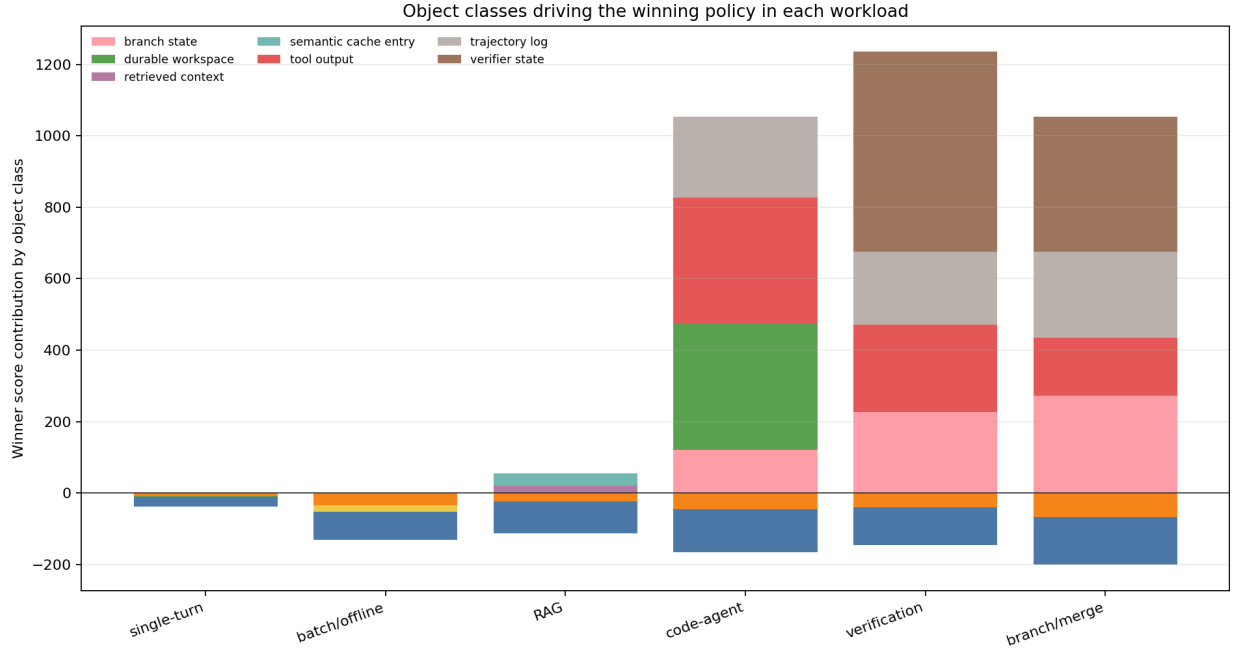


Figure 2: Object-level score attribution from the simulator. The figure separates KV-dominated controls from agentic regimes where durable workspace, verifier state, tool output, branch state, and trajectory log contribute materially.

horizon, zero correctness loss, and zero recomputation cost did not erase all policy differences because other mechanisms remained nonzero. Context-cap saturation with a positive durable horizon preserved durable-workspace differences, matching the earlier lifetime-model claim that durable state is not bounded by the KV context window.

The cycle 4 audit found no critical issues. The moderate issue was the unused energy and dollar proxy fields. After patching, the auditor reran the simulator and plot scripts, verified both figures as nonblank at 1920×1040 , checked required columns and special cases, confirmed controls and non-KV agentic drivers, and appended the M-SIM-1 ledger event.

Cycle 5: Scheduling Abstraction Evaluation

Cycle 5 built `memory-centric-agentic/scheduling_abstractions.md`, `scripts/evaluate_scheduling_abstractions.py`, `scripts/plot_scheduling_abstractions.py`, and six generated outputs: `scheduling_unit_comparison.csv`, `scheduling_regime_winners.csv`, `scheduling_failure_modes.csv`, `scheduling_special_cases.csv`, `scheduling_abstraction_plot.png`, and `scheduling_failure_modes.png`.

The central idea was that a scheduler can only make useful placement decisions if its unit of scheduling exposes the variables that determine object lifetime, reuse, movement, and correctness risk. The evaluator compared eight units: request, job, kernel, model, cache page, context segment, memory object, and agent trajectory DAG.

The scoring model was:

$\text{Benefit}(\text{unit}, \text{workload}) = \text{observed retained value} + \text{movement avoidance} + \text{reuse capture} + \text{branch capture} + \text{durability capture} + \text{correctness capture} - \text{coordination overhead}$

This is a synthetic score, not a measured latency or cost. Its purpose is to identify which information boundary is sufficient for each workload.

The validated winners were:

Workload regime	Preferred scheduling unit	Runner-up	Mechanism label	Dominant object
single-turn chat control	model	cache page	coarse serving boundary	prefix cache
batch summarization/offline inference control	model	cache page	coarse serving boundary	intermediate scratch
RAG with retrieved-context reuse	memory object	agent trajectory DAG	context reuse boundary	semantic cache entry
code-agent loop with tool outputs and durable workspace	agent trajectory DAG	memory object	trajectory branch/verifier boundary	durable workspace
verification-heavy agent	agent trajectory DAG	memory object	trajectory branch/verifier boundary	verifier state
multi-agent branch/merge run	agent trajectory DAG	memory object	trajectory branch/verifier boundary	verifier state

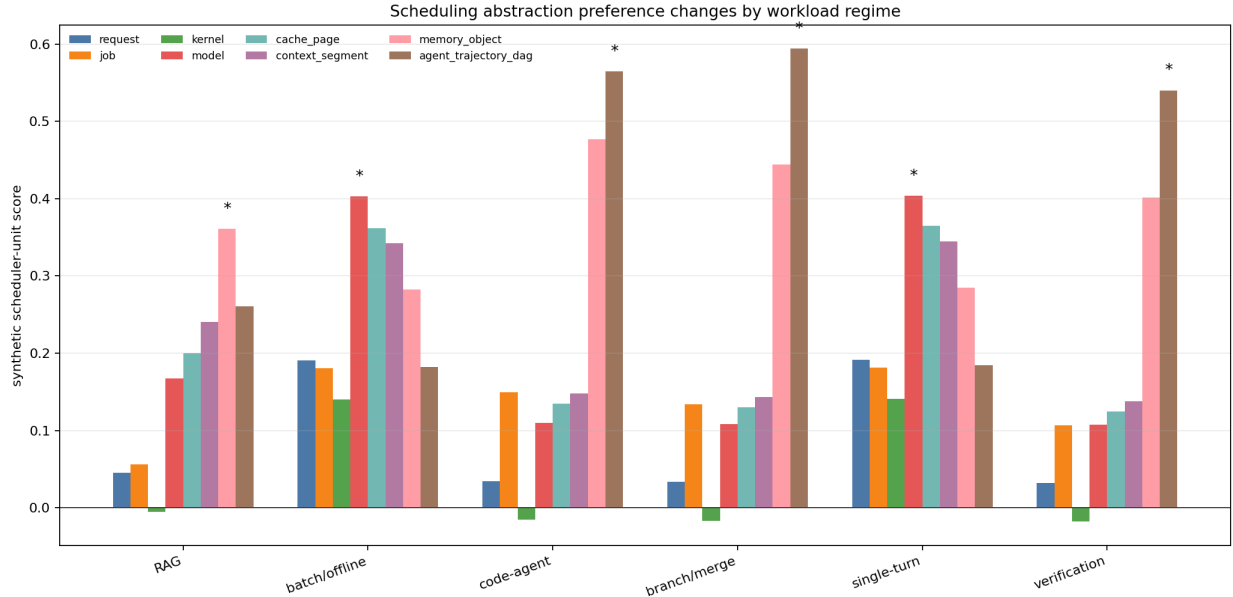


Figure 3: Scheduling abstraction winners by workload. Controls remain at coarse model/cache-page boundaries, RAG moves to memory-object scheduling, and branch/verifier/durable regimes move to trajectory-DAG scheduling.

The failure-mode table explains what each losing abstraction cannot observe. For example, request and kernel scheduling miss object class, reuse probability, correctness sensitivity,

durability horizon, verifier delay, and branch survival. Memory-object scheduling can observe object-local reuse and loss, but it cannot observe cross-object branch/merge dependencies. That is why it is sufficient for RAG but loses to trajectory-DAG scheduling for code-agent, verification-heavy, and multi-agent branch/merge workloads.

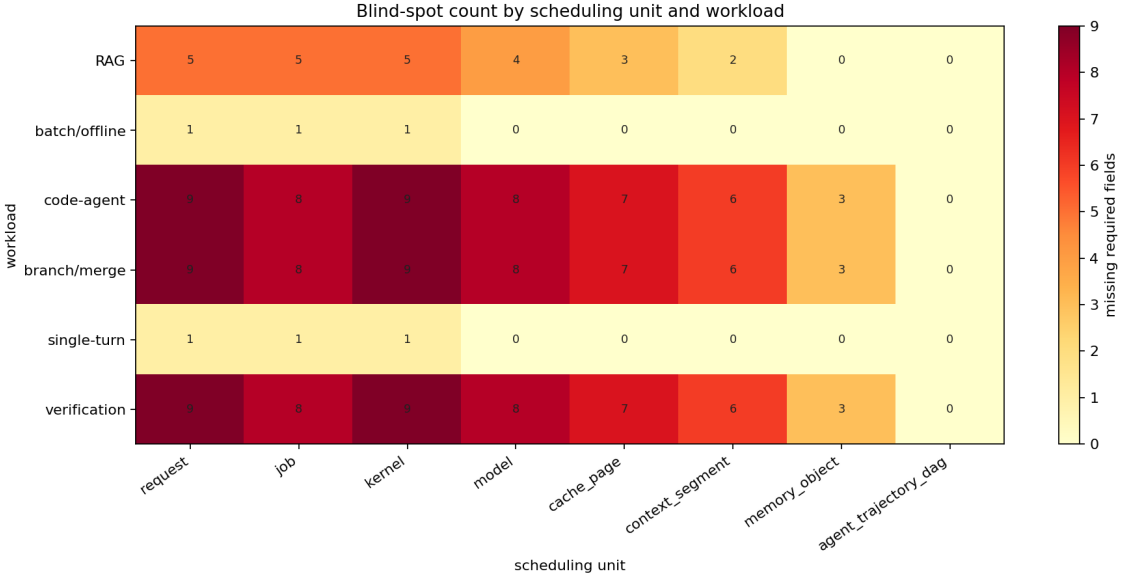


Figure 4: Scheduling failure-mode summary. The rows identify where insufficient visibility causes a unit to evict or recompute state without observing the relevant lifetime boundary.

The scheduling special cases preserved the simulator’s mechanism checks. Equal tier costs collapsed movement-driven differences. No reuse collapsed context/object reuse advantages but left correctness-sensitive verifier and durable mechanisms. Context-cap saturation with a positive durable horizon preserved trajectory and durable workspace value.

The cycle 5 audit found no critical or moderate issues. It reran the evaluator and plot generator, regenerated 48 comparison rows, 6 winner rows, 33 failure rows, and 6 special-case rows, verified both figures as nonblank, compiled the scripts, and ran the project validators. The audit decision was validated.

Cycle 6: Architecture Synthesis

Cycle 6 built `memory-centric-agentic/architecture_proposal.md`, `scripts/synthesize_architecture_package.py`, `scripts/plot_architecture_synthesis.py`, and seven generated outputs: `architecture_options.csv`, `runtime_compiler_hook_matrix.csv`, `architecture_policy_matrix.csv`, `architecture_failure_modes.csv`, `research_agenda_ranked.csv`, `architecture_option_matrix.png`, and `runtime_hook_coverage.png`.

The architecture proposal defined a three-level compatibility stack.

Option A, `A_conventional_request_model_kv_serving`, is the default path for ordinary serving. It targets single-turn and batch/offline controls, uses model or cache-page scheduling, and supports weights, KV cache, prefix cache, and intermediate scratch. Its benefit is low overhead. Its failure mode is overextension: if used for deeper agentic regimes, it misses durable workspace, verifier state, tool output, and trajectory-log value.

Option B, `B_memory_object_aware_runtime`, adds a memory-object registry and object-local reten-

tion policy. It targets RAG-style context reuse and supports semantic cache entries, retrieved context, tool output, and prefix cache. It requires fields such as object ID, object class, size, reuse probability, recompute cost, correctness sensitivity, and provenance pointer. Its failure mode is missing cross-object branch/merge dependencies.

Option C, `C_trajectory_dag_memory_fabric`, treats the agent run as a graph of actions, branches, verifier loops, merges, discards, replay points, and durable artifacts. It targets code-agent, verification-heavy, and multi-agent branch/merge regimes. It supports durable workspace, verifier state, tool output, branch state, trajectory log, and KV cache. Its failure mode is metadata and coordination overhead when applied to simple workloads.

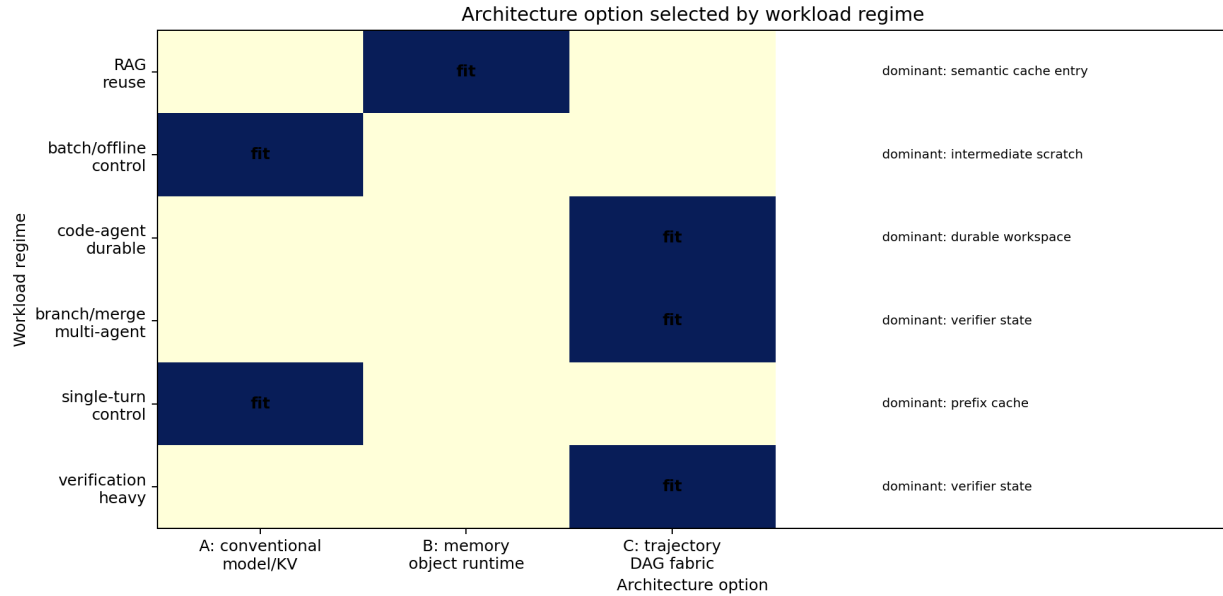


Figure 5: Architecture option matrix. The figure maps workload regimes to the three architecture options and annotates the dominant memory object behind each selection.

The runtime/compiler hook matrix defined 12 hooks:

Hook	Primary purpose
<code>object_registry</code>	Make memory objects addressable by class and size.
<code>lifetime_boundary</code>	Expose object start and end boundaries.
<code>reuse_probability_estimator</code>	Distinguish reusable objects from dead state.
<code>retention_value_estimator</code>	Compare reuse/loss value against residency and movement cost.
<code>correctness_sensitive_pin</code>	Prevent correctness-sensitive evidence from being treated as latency-only cache state.
<code>durability_horizon</code>	Keep durable artifacts until downstream dependencies expire.
<code>branch_state_annotation</code>	Preserve branch-local state according to fanout, survival, and merge probability.
<code>verifier_retention_barrier</code>	Retain tests, counterexamples, and verifier traces through validation loops.

Hook	Primary purpose
<code>trajectory_graph_edge</code>	Connect object-local state to branch, merge, discard, replay, and resume dependencies.
<code>tier_placement_hint</code>	Expose candidate tiers and transfer-cost proxies.
<code>compression_boundary</code>	Separate compressible summaries from raw pointers needed for replay or provenance.
<code>provenance_pointer</code>	Track source URI, version, and invalidation signal.

The policy matrix mapped all 11 memory-object classes to tiering, retention, compression, re-computation, provenance, and eviction-failure strategies. A key result is that eviction failure is not always a cache miss. For retrieved context, it can mean stale evidence or lost citation provenance. For verifier state, it can mean missed counterexamples or repeated validation. For trajectory logs and durable workspace, it can mean broken replay, lost auditability, or missing deliverables.

The architecture failure-mode table gave seven falsification tests:

- Controls require trajectory/DAG hooks.
- RAG is overstated as trajectory evidence.
- The benefit reduces to KV cache scaling.
- Synthetic weights alone drive all conclusions.
- Coordination overhead dominates retained value.
- Semantic reuse becomes stale or unverifiable.
- Durable workspace becomes unbounded storage.

The ranked research agenda had 10 items. The top five were trace-calibrated object lifetime studies, coordination-overhead benchmarks for object versus DAG scheduling, calibrated retention-value sweeps with measured tier costs, RAG semantic-cache invalidation experiments, and verifier-state pinning experiments for code agents.

The cycle 6 audit found no critical issues and one moderate issue. The original `architecture_options.csv` listed supported objects only from dominant scheduler outputs, which underreported retrieved context and tool-output support for Option B and branch, trajectory, and tool-output support for Option C. The auditor patched `scripts/synthesize_architecture_package.py` to give each architecture option an explicit supported-object set while retaining observed dominant objects from scheduling results. After the patch, the audit reran synthesis, script compilation, plot generation, figure checks, semantic probes, `promise_check`, and `org_check`. The final decision was validated.

Discussion

Cycles 4-6 converted the project from model-building into architecture selection. The strongest supported design principle is:

Expose the memory-state variables that are causal for the workload, and stop at the coarsest boundary that preserves them.

This principle explains all three architecture options. Conventional serving is still the right boundary for controls because agentic metadata would add coordination overhead without exposing useful state. Memory-object scheduling is the right boundary for RAG because retrieved context and semantic cache entries need object identity, reuse, provenance, and

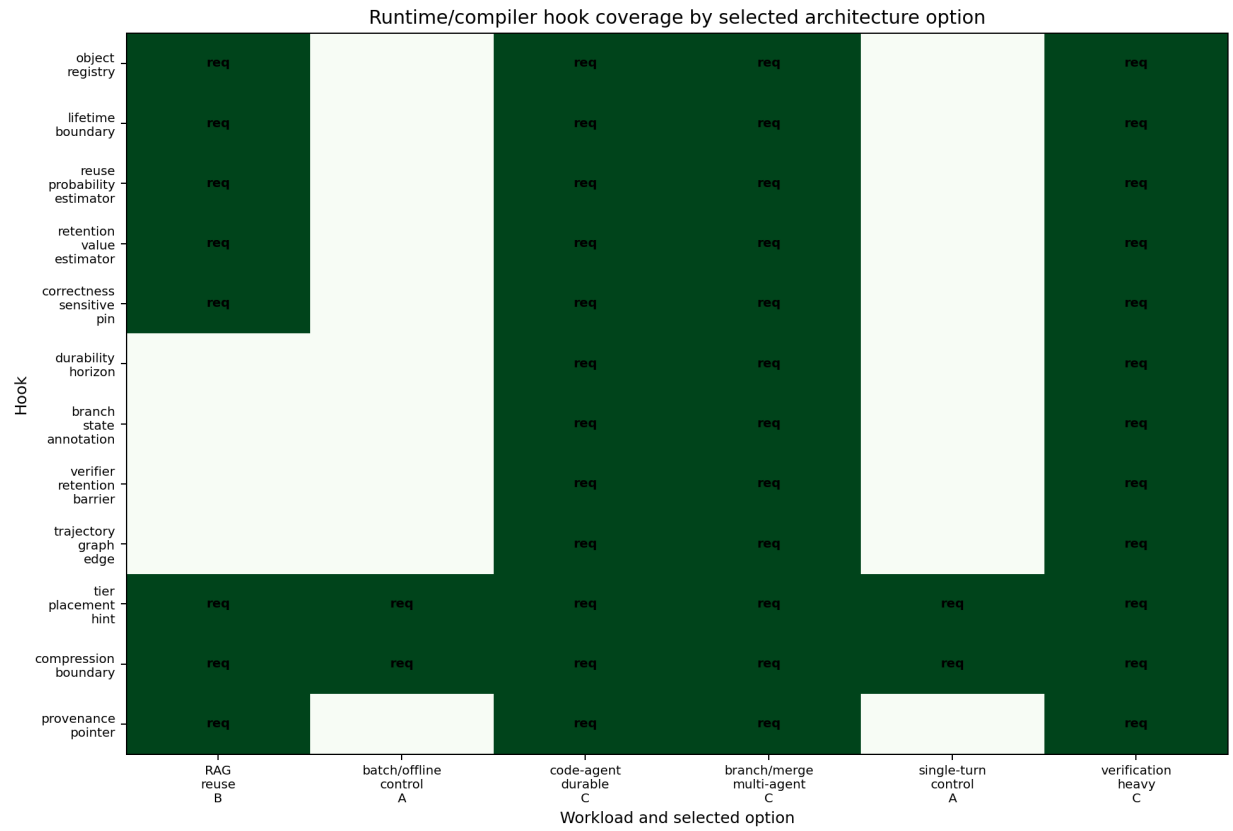


Figure 6: Runtime and compiler hook coverage. The figure shows which hooks are required by each workload regime and architecture option.

invalidation, but not necessarily branch graph structure. Trajectory-DAG scheduling is justified only when future value depends on branch survival, verifier delay, merge/discard state, trajectory logs, or durable workspace dependencies.

The result is not a generic survey claim that memory always dominates compute. It is a conditional architecture claim. In the synthetic artifacts, the thesis is weakened for single-turn chat and batch/offline inference, ambiguous for RAG, and strengthened for code-agent, verification-heavy, and branch/merge regimes. This distinction prevents the architecture proposal from becoming a universal prescription.

The main evidentiary limits are explicit. The workload events are synthetic. Tier costs are normalized proxies. Energy and dollar scores are synthetic placeholders. There is no queueing model, production arrival process, multi-tenant contention model, measured compression overhead, or vendor hardware constant. These omissions do not invalidate the cycle outputs; they define the next evidence required before the package can support measured latency, power, or cost claims.

Conclusions and Recommendations

Cycles 4-6 completed the planned chain from simulator to scheduler to architecture proposal.

The validated architecture is a three-level stack:

1. Keep conventional request/model/KV-centric serving as the low-overhead compatibility path.
2. Add a memory-object-aware runtime when object-local reuse, provenance, invalidation, recomputation cost, or correctness sensitivity becomes causal.
3. Add a trajectory/DAG-aware memory fabric only when branch survival, verifier delay, merge/discard state, trajectory logs, or durable workspace dependencies determine retained value.

The next work should follow the ranked agenda rather than repair these milestones. The highest-priority next step is trace calibration: measure object lifetimes, reuse distances, branch survival, verifier delay, durable horizon, and recomputation/correctness costs in realistic agentic runs. The second priority is coordination overhead: measure whether object and DAG metadata updates erase the retained-value gains shown in synthetic form. The third priority is calibrated tier modeling: replace normalized memory-tier placeholders with measured latency, bandwidth, capacity, energy, and cost values.

References

No external references are cited in this cycle report. REFERENCES.md was not present in the workspace during report assembly, and the cycle artifacts explicitly avoided introducing unsourced hardware constants, production trace claims, measured datacenter prices, or vendor-specific values.

Appendix: Implementation Details

Code Organization

Cycle 4 simulator artifacts:

Artifact	Purpose
memory-centric-agentic/simulator_design.md	Defines simulator scope, consumed inputs, event schema, policies, metrics, special cases, and limitations.
scripts/simulate_memory_policies.py	Generates synthetic workload events and policy scores.
scripts/plot_sim_policy_results.py	Renders policy result and object-breakdown figures.
data/sim_workload_events.csv	215 generated synthetic events plus header.
data/sim_policy_results.csv	24 policy/workload rows plus header.
data/sim_policy_object_breakdown.csv	112 object-contribution rows plus header.
data/sim_special_cases.csv	7 special cases plus header.
data/sim_policy_results.png	Policy comparison figure, 1920 x 1040.
data/sim_object_breakdown.png	Object attribution figure, 1920 x 1040.

Cycle 5 scheduling artifacts:

Artifact	Purpose
memory-centric-agentic/scheduling_abstractions.md	Defines scheduling units and expected collapses.
scripts/evaluate_scheduling_abstractions.py	Scores eight scheduling units across six workload regimes.
scripts/plot_scheduling_abstractions.py	Renders scheduling winner and failure-mode figures.
data/scheduling_unit_comparison.csv	48 comparison rows plus header.
data/scheduling_regime_winners.csv	6 winner rows plus header.
data/scheduling_failure_modes.csv	33 failure rows plus header.
data/scheduling_special_cases.csv	6 special cases plus header.
data/scheduling_abstraction_plot.png	Scheduling winner figure, 2080 x 1040.
data/scheduling_failure_modes.png	Failure-mode figure, 1920 x 928.

Cycle 6 architecture artifacts:

Artifact	Purpose
memory-centric-agentic/architecture_proposal.md	Defines architecture options, responsibilities, tier mapping, policy surface, failure criteria, and agenda.
scripts/synthesize_architecture_package.py	Generates architecture option, hook, policy, failure-mode, and agenda tables.
scripts/plot_architecture_synthesis.py	Renders architecture option and hook coverage figures.
data/architecture_options.csv	3 architecture options plus header.
data/runtime_compiler_hook_matrix.csv	12 runtime/compiler hooks plus header.
data/architecture_policy_matrix.csv	11 object-policy rows plus header.
data/architecture_failure_modes.csv	7 failure modes plus header.
data/research_agenda_ranked.csv	10 ranked agenda items plus header.
data/architecture_option_matrix.png	Architecture option figure, 1980 x 990.

Artifact	Purpose
data/runtime_hook_coverage.png	Runtime/compiler hook figure, 1980 x 1350.

The workspace manifest was updated at `MANIFEST.md` to reflect cycles 1-6. It records 10 scripts, 2,074 script lines, 8 Markdown model/assumption/proposal files, 23 CSV data/model files, 9 figures, and 6 completed sub-topics.

Validation Summary

Cycle 4 validation, session 87c49730-6591-4fac-a9f5-62f2630baf6a:

- Critical findings: none.
- Moderate finding: energy and dollar proxy fields were checked but not used in simulator scoring.
- Fix: patched `scripts/simulate_memory_policies.py` to derive workload-level proxy scales from `energy_proxy_score` and `dollar_proxy_score`.
- Reruns: simulator, plot generator, schema checks, special-case checks, figure checks, promise check, organization check.
- Decision: validated.

Cycle 5 validation, session eb34f7da-0f9f-498f-9b3a-37bf33ec243e:

- Critical findings: none.
- Moderate findings: none.
- Minor process note: first ledger append lacked `event_id`; retry succeeded.
- Reruns: scheduling evaluator, plot generator, script compilation, promise check, organization check.
- Decision: validated.

Cycle 6 validation, session c8733e5d-7874-4e7c-aae1-e679c3897573:

- Critical findings: none.
- Moderate finding: architecture option supported-object sets were underreported because they came only from dominant scheduler outputs.
- Fix: patched `scripts/synthesize_architecture_package.py` to give each option an explicit supported-object set.
- Reruns: synthesis script, script compilation, plot generator, figure checks, independent semantic probe, promise check, organization check.
- Decision: validated.

Session References

Cycle	Role	Session ID	Contribution
4	researcher	89a82512-72e9-4051-bc4e-79e8086a63db	Defined M-SIM-1, the workload generator and memory-policy harness.

Cycle	Role	Session ID	Contribution
4	worker	8a332843-0be4-4bca-8d84-85f22b215bd5	Built simulator design, event generator, policy scorer, plots, and outputs.
4	auditor	87c49730-6591-4fac-a9f5-62f2630baf6a	Patched proxy-score consumption and validated M-SIM-1.
5	researcher	ceac8c77-51f8-4162-89ee-72b733822019	Defined M-SCHED-1, the scheduling abstraction evaluation.
5	worker	5468f214-6855-4f34-8aad-1613ec95000e	Built scheduling evaluator, plots, winners, failures, and special cases.
5	auditor	eb34f7da-0f9f-498f-9b3a-37bf33ec243e	Validated M-SCHED-1.
6	researcher	7031eb0d-9bee-407f-bc44-57409388096b	Defined M-ARCH-1, the architecture synthesis milestone.
6	worker	94a84a42-0e3f-4bf8-8f4f-184625a04097	Built architecture proposal, synthesis script, tables, agenda, and plots.
6	auditor	c8733e5d-7874-4e7c-aae1-e679c3897573	Patched supported-object sets and validated M-ARCH-1.

Cross-Reference Map

Source artifact	Downstream artifact	Relationship
memory-centric-agentic/memory_objects.csv	scripts/simulate_memory_policies.py	Memory object classes become event object classes.
memory-centric-agentic/workload_classes.csv	scripts/simulate_memory_policies.py	Workload classes become simulator regimes and controls.
memory-centric-agentic/lifetime_parameters.csv	scripts/simulate_memory_policies.py	Lifetime and reuse fields shape event parameters.
data/cost_model_scenarios.csv	scripts/simulate_memory_policies.py	Scenario sizes, retained-value scores, energy proxies, dollar proxies, and thesis labels seed simulator scoring.

Source artifact	Downstream artifact	Relationship
data/sim_policy_results.csv	scripts/evaluate_scheduling_abstractions.py	Policy winners and dominant object classes drive scheduler comparison.
data/sim_policy_object_breakdown.csv	scripts/evaluate_scheduling_abstractions.py	Object-level contributions determine which scheduling fields are required.
data/scheduling_regime_winners.csv	scripts/synthesize_architecture_package.py	Scheduling winners map workloads to architecture options.
data/scheduling_failure_modes.csv	scripts/synthesize_architecture_package.py	Scheduler visibility failures feed architecture failure criteria and hook requirements.
scripts/synthesize_architecture_package.py	memory-centric-agent/architecture_proposal.md	Generated tables and validated prior milestones support the final architecture proposal.